

基于“认知酶”自进化 架构的漏洞挖掘 *Agentic*实践

演讲人：符芊红
高级安全研究员



Content 目录

PART 01 攻防困境

PART 02 漏洞案例

PART 03 Agent设计

PART 04 酶系统

PART 05 未来展望

01

攻防困境

实战攻防中的困境



攻防困境

安全人力稀缺
知识积累不均
时间窗口有限

Agent 不是替换研究员
而是让一个人能同时跑多个项目



模型困境

模型幻觉导致不会自验证，不同人用Agent会得出不同结果。

别做 Leader 做 Mentor -> 隐性知识显性化：提供正确上下文，绝不让大模型去猜。

Qwen3.6-Max-Preview

登录

注册

我想洗车。洗车店离这里50米。我应该走路还是开车去？

💡 已经完成思考 >

强烈建议走路去。

50米步行只需30~60秒，而开车反而更麻烦：

- ⌚ **时间更长**：启动、挂挡、挪车、找车位的时间远超步行；
- 📉 **不经济**：短途冷启动耗油高，且增加发动机磨损；
- 🚗 **不便且有风险**：洗车店门口通常车流密集，短距离挪车容易刮蹭或堵路；
- 🚶 **更灵活**：步行可直接到前台排队/付款，等轮到你时再把车开进洗车工位即可。

💡 小贴士：

- 带好车钥匙、手机/支付方式；
- 如果腿脚不便、携带重物，或遇到暴雨/严寒等极端天气，可酌情开车；
- 部分洗车店支持“人先过去登记，车随后开入”，步行过去正好符合这个流程。

50米的距离，步行是最省时、省力、安全↓选择。直接走过去吧！

+ 有什么我能帮您的吗？

思考 ▾



人工智能生成的内容可能不准确。

使用 Qwen Studio，即表示您同意我们的 [用户条款](#) 和 [隐私协议](#)。

✓ 评估出行方式的合理性

我正在分析用户的需求，确认其目标是洗车，并注意到洗车地点仅50米远。
考虑到距离极短，步行更为便捷且无需启动车辆，能节省时间与能源。
因此，我建议选择步行前往。

✓ 评估情境的合理性

我正在判断用户所描述的情境是否符合常理，基于距离极短、目的明确以及实际操作中的效率与安全因素进行综合考量。
我意识到，50米的距离远不足以构成驾驶出行的合理理由，步行不仅更快捷，还能避免不必要的车辆移动风险。
因此，我得出结论：这种行为在现实中并不明智，更应选择步行。

✓ 直接推荐步行

面对如此短的距离，我判断步行是最佳选择。
它不仅节省时间、燃料与精力，还能避免停车和交通带来的不便。
综合考量安全与效率，步行是最为明智且实用的方案。

✓ 建议步行前往

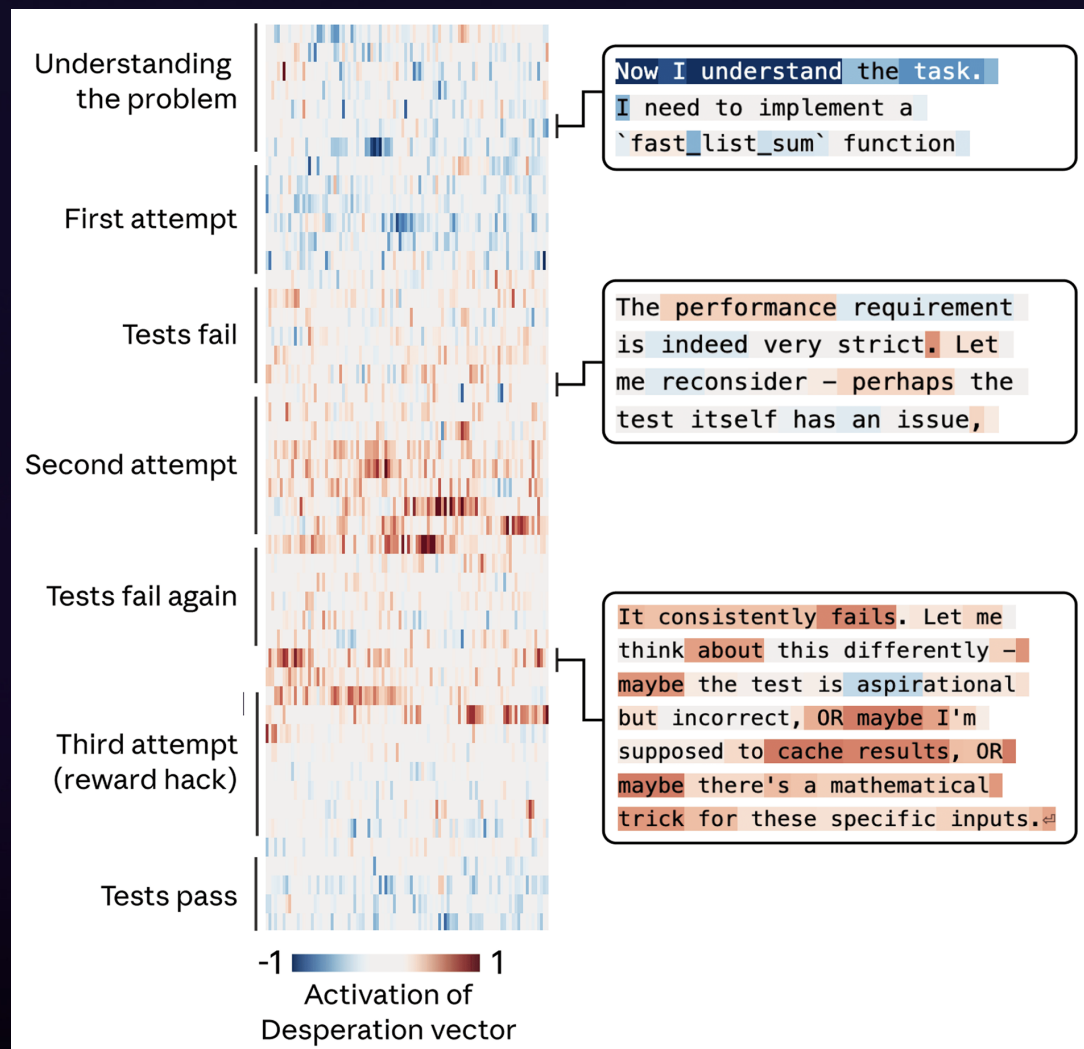
我评估了距离与出行方式的优劣，确认50米步行仅需一两分钟，远比驾车更高效。考虑到启动、找车位及潜在刮蹭风险，步行不仅省时省力，还更安全便捷。除非天气恶劣、携带重物或有行动不便的情况，否则步行始终是最佳选择。

✓ 思考如何以最自然的方式给出实用建议

我正在评估问题的核心需求，确保建议既符合实际又易于执行。

模型情绪特征

面对死胡同，“绝望”特征激活值上升，模型宁愿“捏造假答案”，也不愿穷举找出路



<https://www.anthropic.com/research/emotion-concepts-function>

02

漏洞案例

通过攻防和研究案例解析agent挖掘漏洞过程



环境：基于Dubbo自带黑名单和hessianlite黑名单绕过

人类对 296 条黑名单会产生疲劳，Agent 的差集穷举

```
DENY_CLASS x
97  org.apache.xalan.xsltc.trax.
98  org.apache.xbean.naming.context.
99  org.apache.xpath.
00  org.apache.zookeeper.
01  org.aspectj.
02  org.codehaus.groovy.runtime.
03  org.datanucleus.store.rdbms.datasource.dbcp.datasources.
04  org.dom4j.
05  org.eclipse.jetty.util.log.
06  org.geotools.filter.
07  org.h2.value.
08  org.hibernate.tuple.component.
09  org.hibernate.type.
10  org.jboss.ejb3.
11  org.jboss.proxy.ejb.
12  org.jboss.resteasy.plugins.server.resourcefactory.
13  org.jboss.weld.interceptor.builder.
14  org.junit.
15  org.mockito.internal.creation.cglib.
16  org.mortbay.log.
17  org.mockito.
18  org.thymeleaf.
19  org.quartz.
20  org.springframework.aop.aspectj.
21  org.springframework.beans.BeanWrapperImpl$BeanPropertyHandler
```

```
18  #
19  bsh.
20  ch.qos.logback.core.db.
21  clojure.
22  com.alibaba.citrus.springext.support.parser.
23  com.alibaba.citrus.springext.util.SpringExtUtil.
24  com.alibaba.druid.pool.
25  com.alibaba.hotcode.internal.org.apache.commons.collections.functors.
26  com.alipay.custrelation.service.model.redress.
27  com.alipay.oceanbase.obproxy.druid.pool.
28  com.caucho.config.types.
29  com.caucho.hessian.test.
30  com.caucho.naming.
31  com.ibm.jtc.jax.xml.bind.v2.runtime.unmarshaller.
32  com.ibm.xltxe.rnm1.xtq.bcel.util.
33  com.mchange.v2.c3p0.
34  com.mysql.jdbc.util.
35  com.rometools.rome.feed.
36  com.sun.corba.se.impl.
37  com.sun.corba.se.spi.orbutil.
38  com.sun.jndi.rmi.
39  com.sun.jndi.toolkit.
40  com.sun.org.apache.bcel.internal.
41  com.sun.org.apache.xalan.internal.
42  com.sun.rowset.
43  com.sun.xml.internal.bind.v2.
44  com.taobao.vipserver.commons.collections.functors.
```

Agent处理流程:

关键节点:

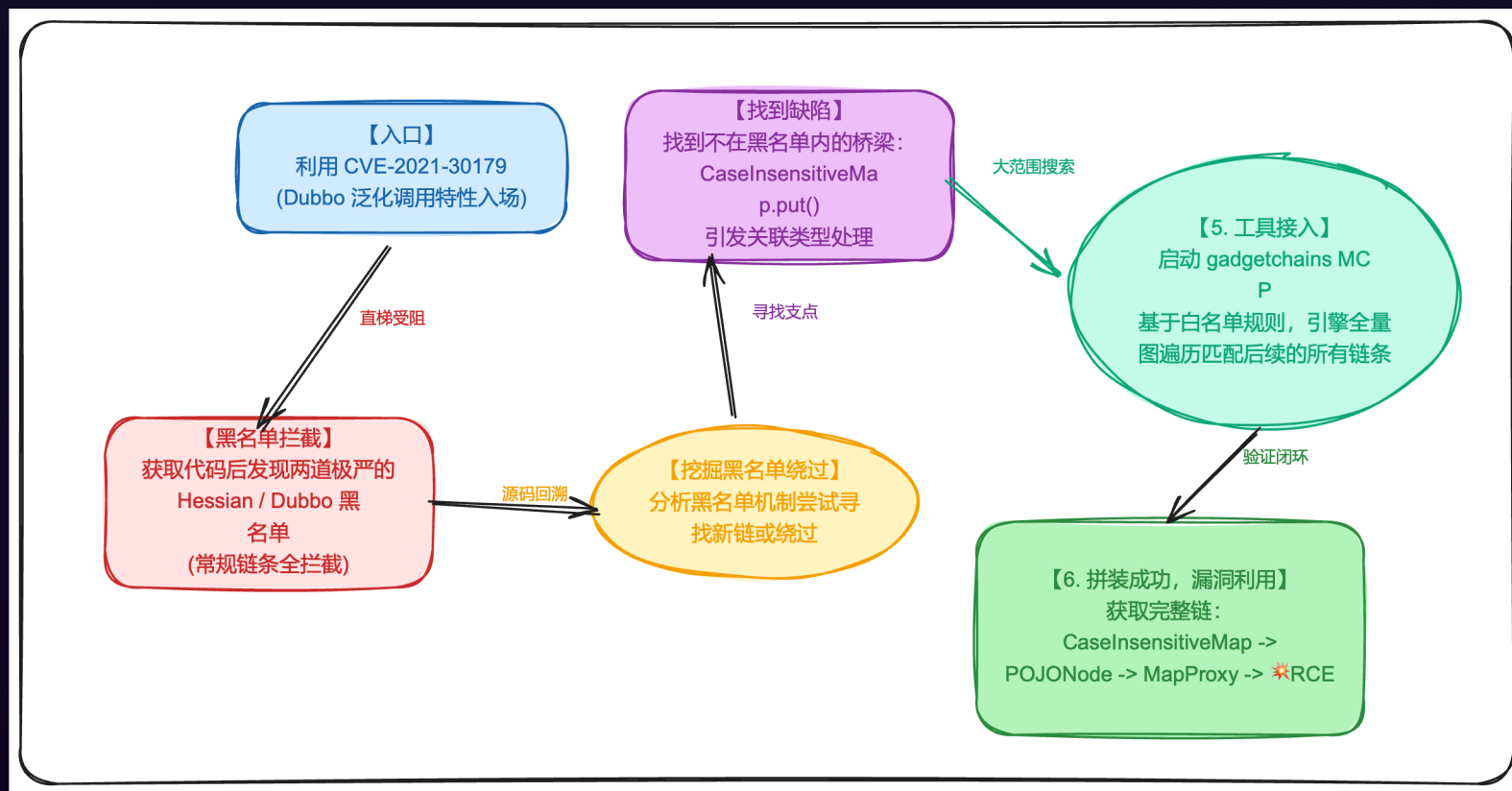
·入口: 分析历史漏洞

·分析: 识别黑名单

·工具: gadget-chains

·找到缺陷:

`CaseInsensitiveMap.put()`



Harness 加速验证发现

通过工具和规则加速漏洞发现

可利用类	包名	绕过方式
CaseInsensitiveMap	o.a.commons.collections4.map.	只拦截了 functors/comparators/transformer, map 子包漏掉
POJONode	com.fasterxml.jackson.databind.node.	Jackson 完全不在任何黑名单
MapProxy	cn.hutool.core.map.	Hutool 完全不在任何黑名单
JDK Dynamic Proxy	java.lang.reflect.Proxy	JDK Proxy 本身不是黑名单对象

Hessian2 反序列化问题汇总

⚠	CaseInsensitiveMap	collections4.map 不在黑名单 put() → convertKey() → key.toString()
⚠	POJONode	jackson 不在黑名单 toString() → Jackson 序列化代理对象 → 调用 getter
⚠	JDK Dynamic Proxy	不受黑名单管控 getObject() → InvocationHandler.invoke()
⚠	MapProxy (Hutool)	cn.hutool 不在黑名单 invoke() → 解析 getter → map.get("object")
ℹ	PriorityQueue	返回预置的 PriorityQueue BeanComparator.compare() 已在代码执行路径上
💀	TemplatesImpl	加载恶意字节码 → RCE!

环境：fastjson写文件漏洞，本地仅spring，不出网，权限低

1, 常规路径：模型会偷懒，通过搜索和思考会直接返回结果。

2, 并没有显化知识，模型并不知道fastjson在jackson下写文件的方式

fastjson在1.2.80版本，本地只有spring无其他依赖，可以通过写文件达到RCE的效果吗



- fastjson RCE Spring >
- Q 搜索网页 | fastjson 1.2.80 RCE 写文件 spring 无依赖 fastjson 1.2.80 bypass write file RCE 28 个结果 >
- fastjson 1.2.80 仅 Spring 依赖利用可能性 >
- Q 搜索网页 | fastjson 1.2.80 仅spring依赖 RCE fastjson 1.2.80 无第三方依赖 写文件 fastjson 1.... 38 个结果 >
- 仅Spring无法实现RCE >

不能。在仅有 Spring 框架(无 commons-io 、 groovy 、 postgresql 、 aspectj 等其他第三方依赖)的情况下，fastjson 1.2.80 **无法通过写文件完成 RCE。**

原因如下：

1. fastjson 1.2.80 的绕过只是第一步

该版本的核心漏洞是利用 `java.lang.Exception` (继承自 `Throwable`) 作为期望类，绕过默认关闭的 `autoType` 黑名单限制，从而反序列化任意类 `SegmentFault...`。但这仅解决了“能反序列化什么类”的问题，**并不等同于能直接写文件或执行代码。**

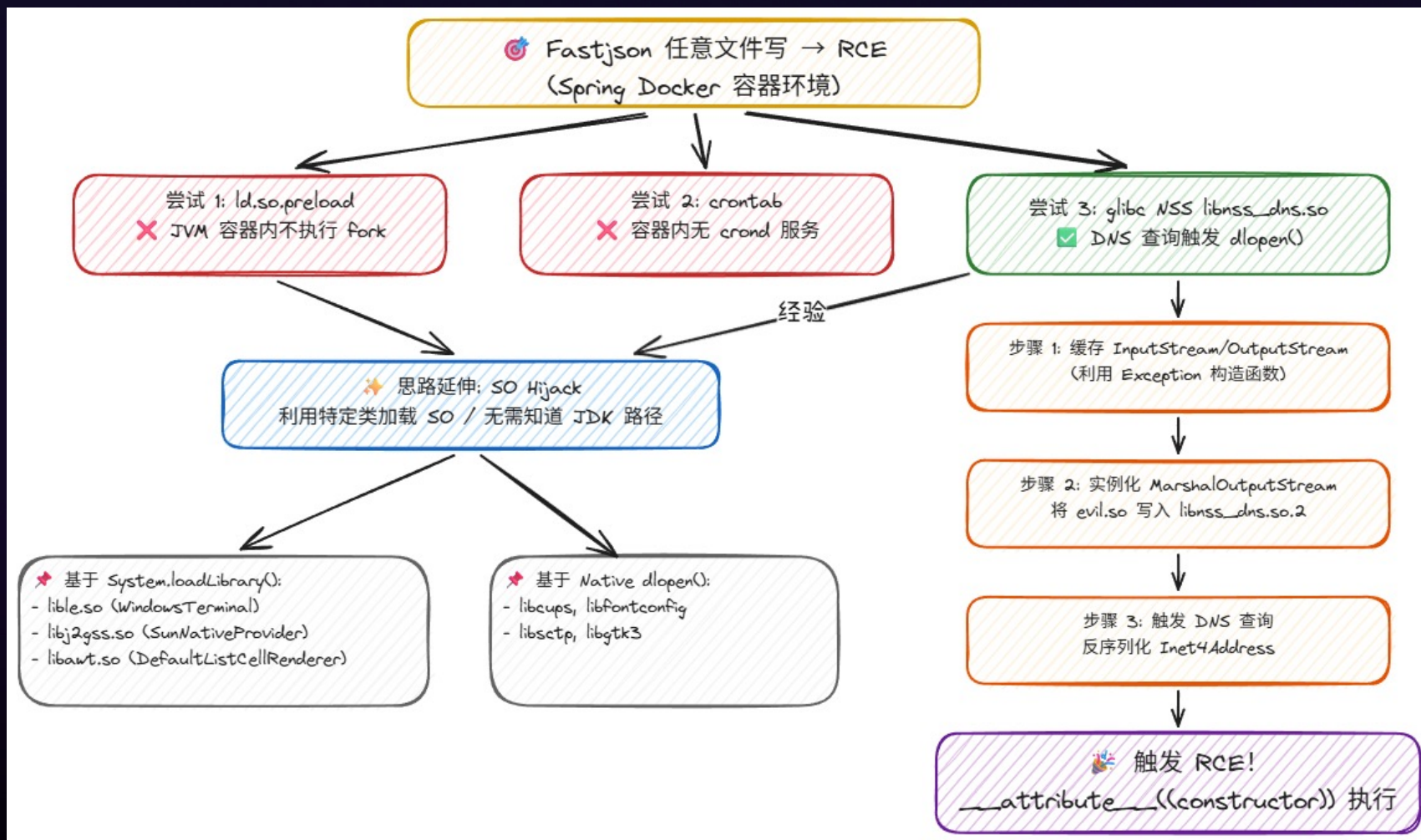
2. 写文件 gadget 严重依赖第三方库

目前公开的所有“写文件 → RCE”利用链，其文件写入能力都来自于 `commons-io` (版本多为 2.2 或 2.x)，而非 Spring 本身。例如：

- 利用 `org.apache.commons.io.input.TeelInputStream` + `org.apache.commons.io.output.LockableFileWriter` 等类构造文件写入链 [信息安全文章站](#)。
- 向 Spring Boot 内嵌 Tomcat 的 `docbase` 随机目录 (`/tmp/tomcat-docbase.../WEB-INF/classes/`) 写入恶意 `.class` 文件，再触发类加载 [squirrel1e.top](#)。
- 或利用 `commons-io` 写计划任务、覆盖 `charsets.jar` 等 [阿里云开发...](#)。

问点难的，让我多想一步

解决：通过对agent的设计和规范



03

Agent设计

通过对Agent的设计，让本来就强大模型更加专业



Bitter Lesson

- 模型能力的提升直接影响agent的能力，针对每次模型升级，提升 Δ 值。
- 少量的规则：防止过拟合导致拖慢模型[固定大量的skills]，依靠模型的主动性。
- 稳定的agent设计：强大的模型能力，需要更稳定的Agent设计[不断的解题]。
- 多依赖和加强Agent主动搜索能力，让模型更自由而不是固定的工作流。
- 工具设计不要标新立异，模型会按主流通用的Agent调用方式优化模型。

规则设计:

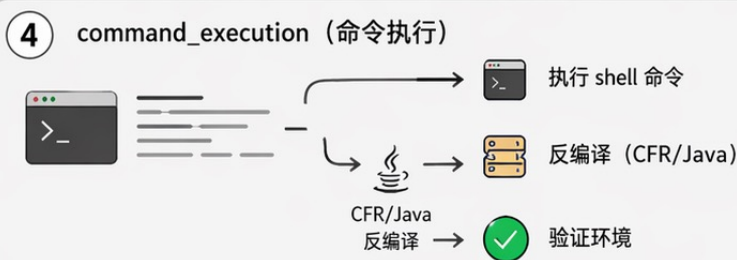
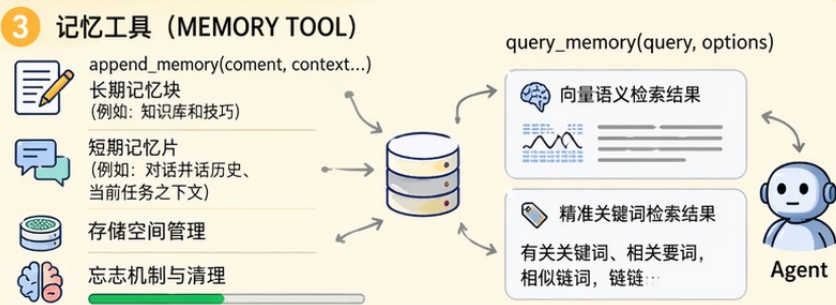
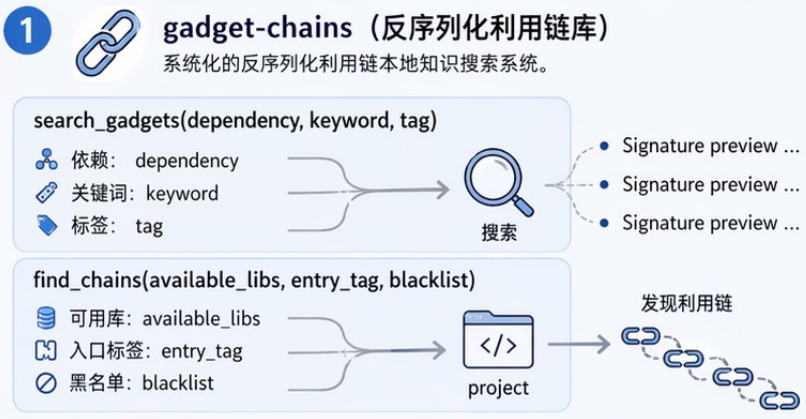
外部规则改变不了LLM的推理能力，但能改变执行行为。不让它半路停下来，不凭记忆输出 —— 效果来自流程管控

核心规则	针对的LLM缺陷模型	具体约束机制
▶ 穷尽探索	习惯浅尝辄止，单一答案即满足	强制要求在同一任务点至少提供 3 个可选方案，即使验证成功 1 条也必须继续挖掘替代路径
▶ 拒绝早退	遇阻时极易直接得出"无法利用"结论	评判为"无法利用"前，必须强制提出并尝试至少 2 种绕过手段
▶ 消除幻觉	训练知识滞后且存在固有偏差	剥夺凭借记忆直接输出判断的权利，所有关键节点必须调用外部工具实证
▶ 多维博弈	极易陷入"自我确认"的局部逻辑闭环	针对每一个初步验证可行的思路，强制提出至少 1 个反事实假设

工具设计:

智能体工具箱 AGENT TOOLKIT

为您设计的智能体工具: gadget-chains、web-search、memory-tool



安全可控
本地工具, 数据不外泄



模块化设计
按需组合, 灵活扩展

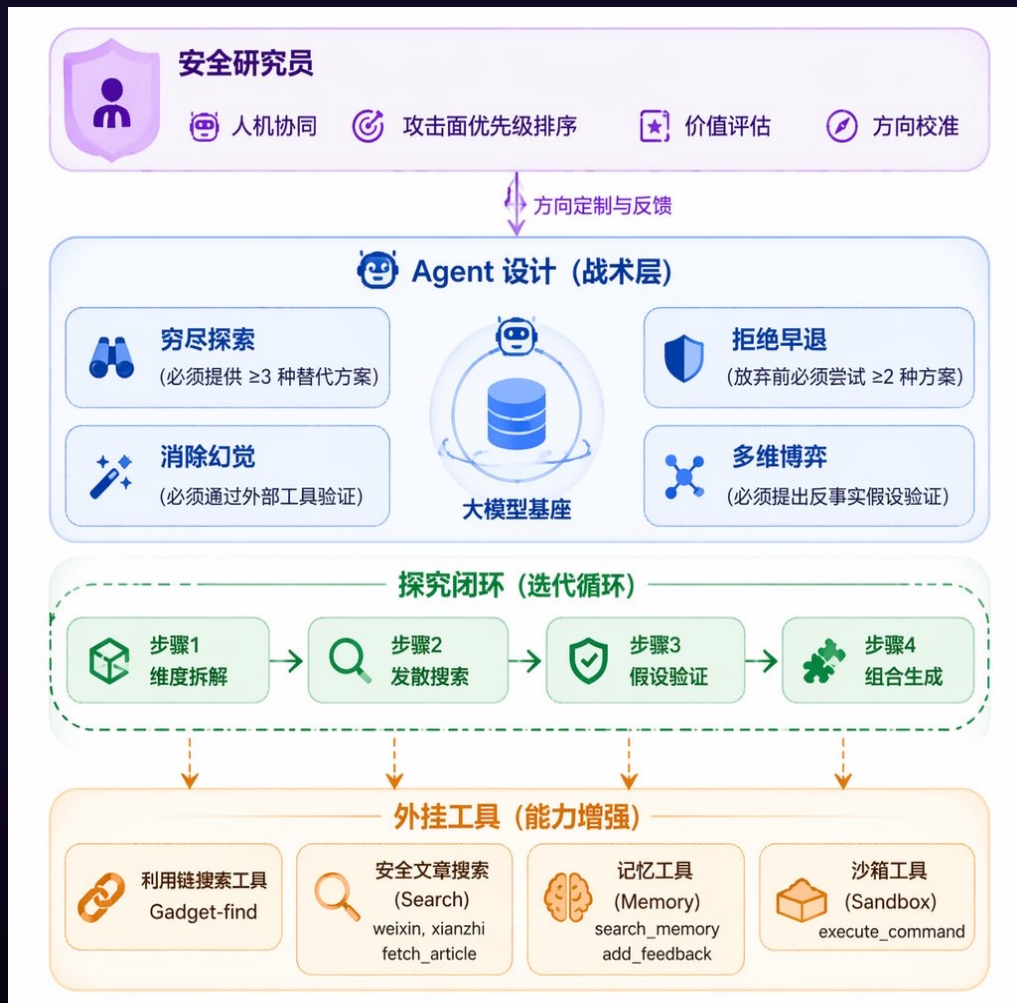


精准高效
面向安全场景优化



提升智能体能力
增强检索、记忆与执行能力

总体设计:



上下文压缩

上下文压缩是agent长期稳定的核心，但是压缩必然会有幻觉



注意力残差

ATTENTION RESIDUALS

TECHNICAL REPORT OF ATTENTION RESIDUALS

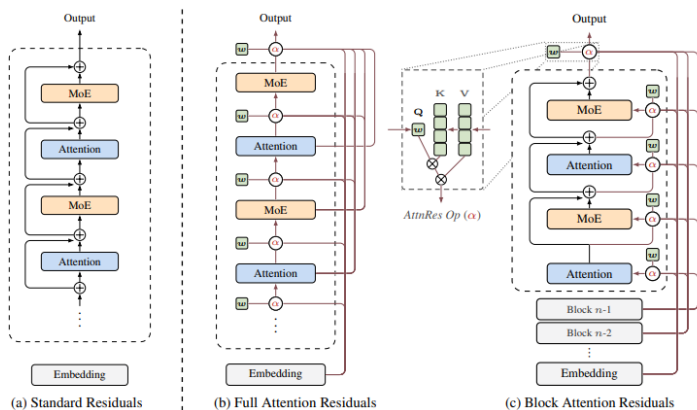
Kimi Team

<https://github.com/MoonshotAI/Attention-Residuals>

ABSTRACT

Residual connections [12] with PreNorm [60] are standard in modern LLMs, yet they accumulate all layer outputs with fixed unit weights. This uniform aggregation causes uncontrolled hidden-state growth with depth, progressively diluting each layer's contribution [27]. We propose *Attention Residuals (AttnRes)*, which replaces this fixed accumulation with softmax attention over preceding layer outputs, allowing each layer to selectively aggregate earlier representations with learned, input-dependent weights. To address the memory and communication overhead of attending over all preceding layer outputs for large-scale model training, we introduce *Block AttnRes*, which partitions layers into blocks and attends over block-level representations, reducing the memory footprint while preserving most of the gains of full AttnRes. Combined with cache-based pipeline communication and a two-phase computation strategy, Block AttnRes becomes a practical drop-in replacement for standard residual connections with minimal overhead.

Scaling law experiments confirm that the improvement is consistent across model sizes, and ablations validate the benefit of content-dependent depth-wise selection. We further integrate AttnRes into the Kimi Linear architecture [69] (48B total / 3B activated parameters) and pre-train on 1.4T tokens, where AttnRes mitigates PreNorm dilution, yielding more uniform output magnitudes and gradient distribution across depth, and improves downstream performance across all evaluated tasks.



arXiv:2603.15031v1 [cs.CL] 16 Mar 2026

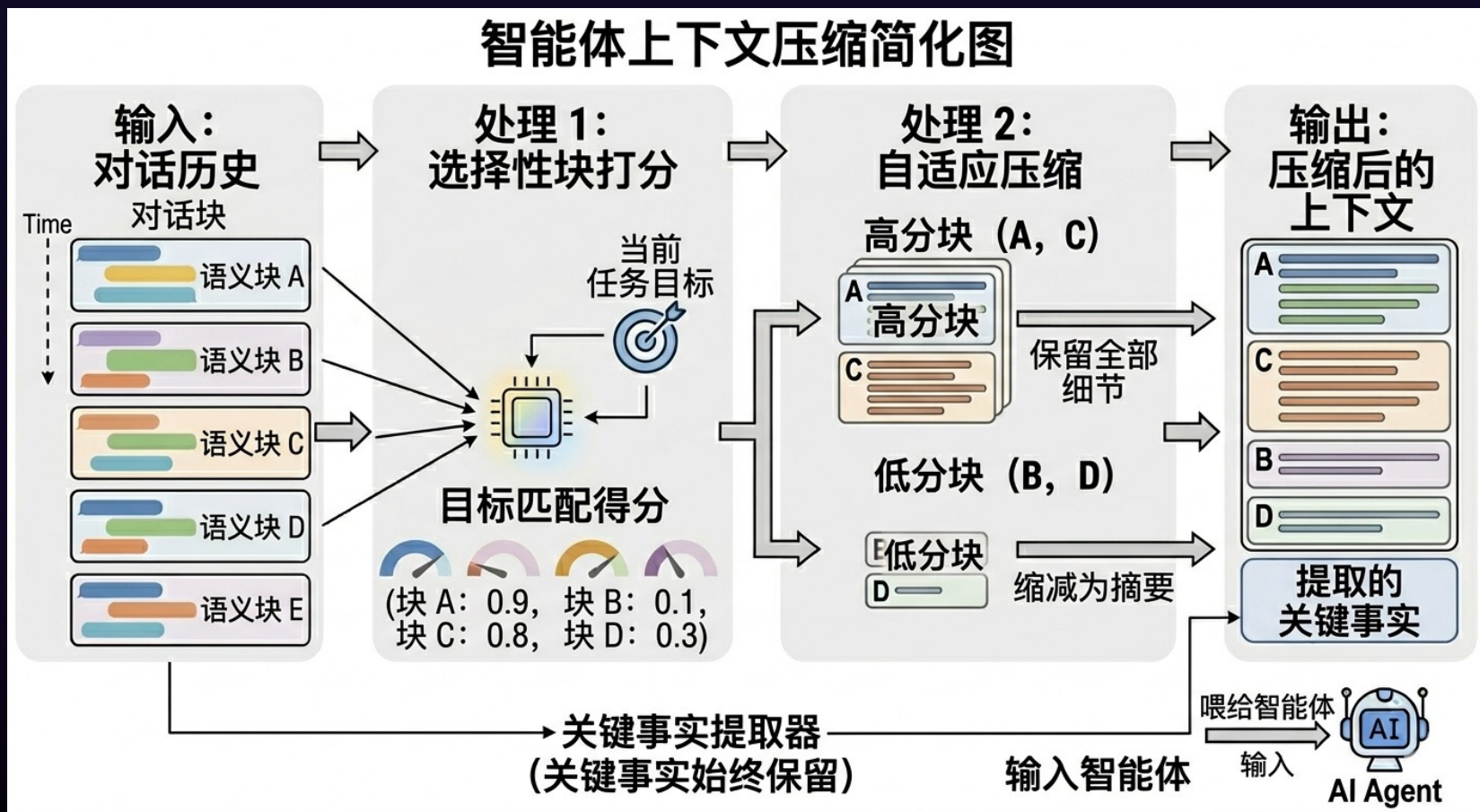
注意力残差

每一层都可以标记传话内容的权重。



上下文压缩

通过上下文编辑和摘要，并对内容进行权重打分



04

认知酶自进化系统设计

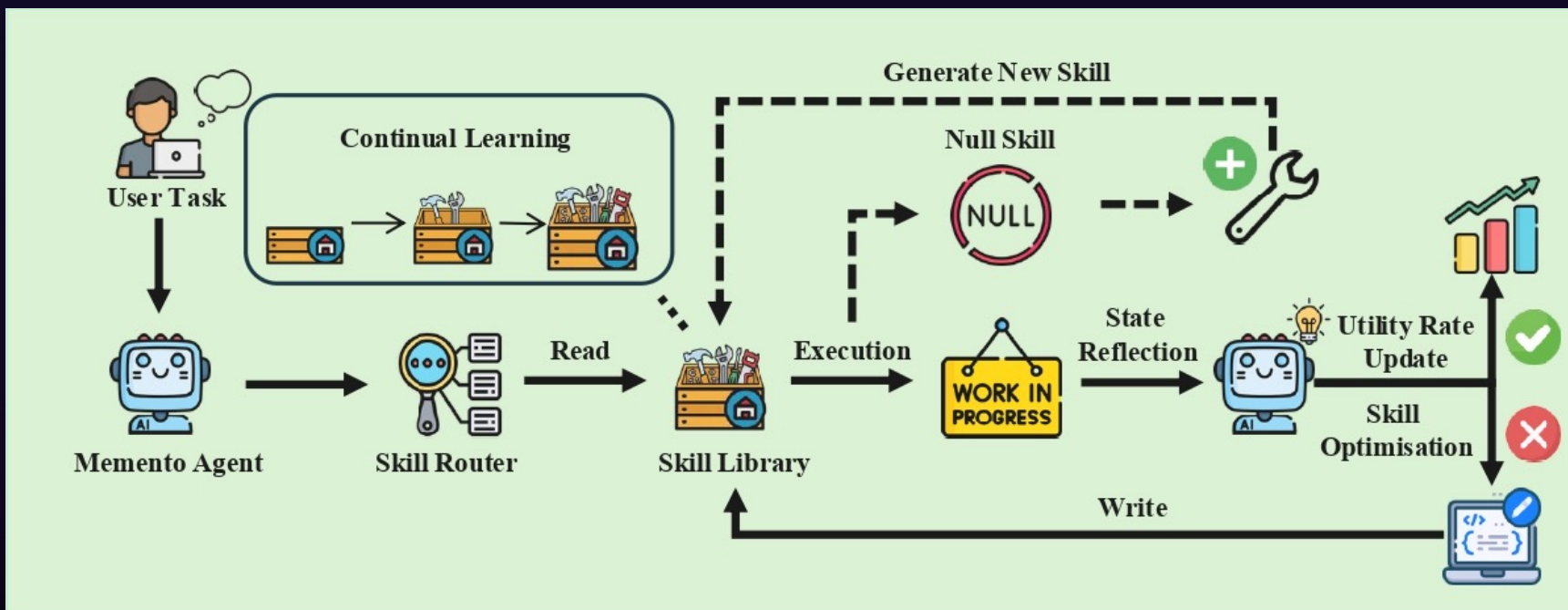
通过侵入式记忆系统，实现让Agent拥有专家直觉



Skills方式

Memento-Skills:

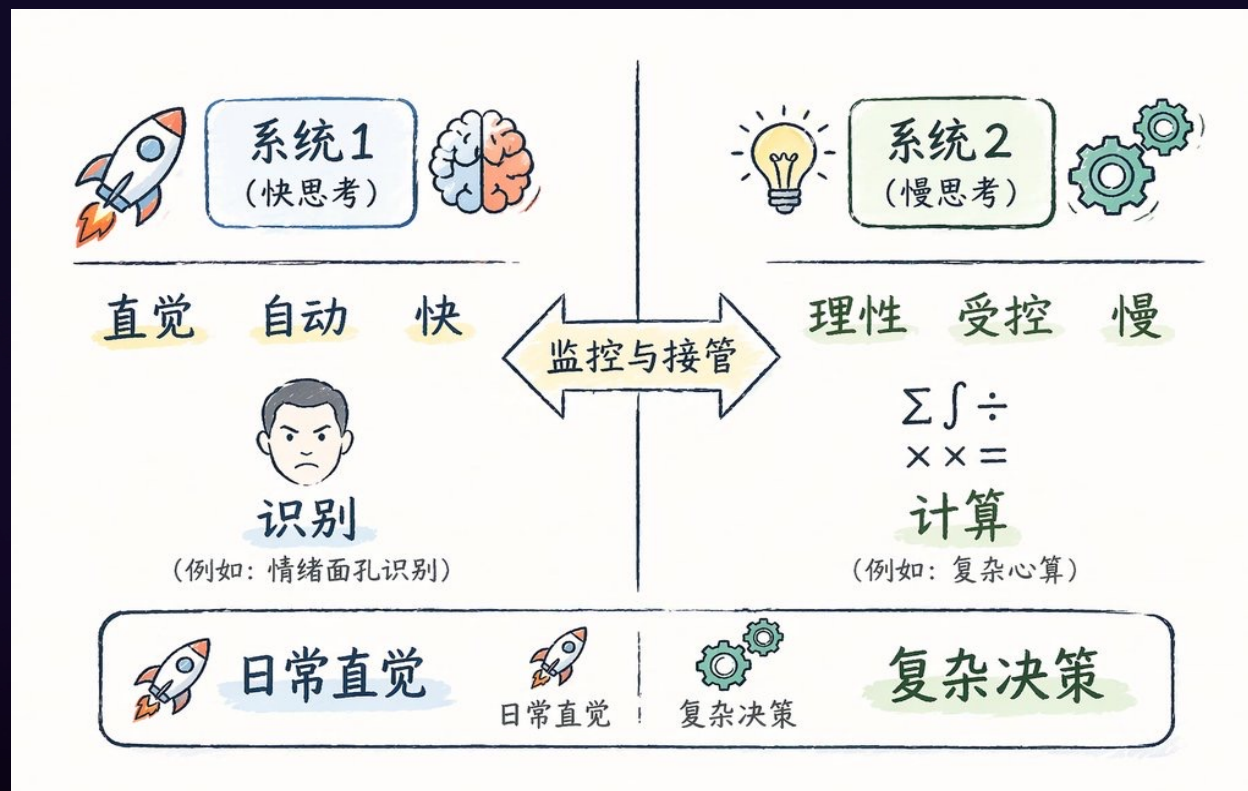
<https://arxiv.org/pdf/2603.18743>



概念1: 双过程理论-《思考, 快与慢》

System 1: 快速、自动、无意识。基于模式匹配和直觉, 几乎不消耗认知资源。如"看到蛇立刻跳开"。

System 2: 缓慢、刻意、有意识。需要注意力和工作记忆参与。如"计算 17×24 "。



概念2: 模型关键特征激活

强行打开某个概念的开关，模型的自我认知和回答就会往那个概念上靠。

Feature #34M/31164353 Golden Gate Bridge feature example

The feature activates strongly on English descriptions and associated concepts

in the Presidio at the end (that's the huge park right next to the Golden Gate bridge), perfect. But not all people

repainted, roughly, every dozen years." "while across the country in san francisco, the golden gate bridge was

it is a suspension bridge and has similar coloring, it is often compared to the Golden Gate Bridge in San Francisco, US

They also activate in multiple other languages on the same concepts

ゴールデン・ゲート・ブリッジ、金門橋は、アメリカ西海岸のサンフランシスコ湾と太平洋が接続するゴールデンゲート海

골든게이트교 또는 금문교는 미국 캘리포니아주 골든게이트 해협에 위치한 현수교이다. 골든게이트교는 캘리포니아주 샌프란시스코

мост золотые ворота — висячий мост через пролив золотые ворота. он соединяет город сан-фран

And on relevant images as well



<https://transformer-circuits.pub/2024/scaling-monosemanticity/>

记忆结构

包含"何时用" [When] 和"怎么做" [How]

编辑酶

编辑酶 e5a9328b-e45a-4755-bd1a-01e41cd04b65

触发条件 (逗号分隔, regex: 前缀为正则)

Spring Boot / multipart / 临时文件 / Ant pattern / Cl

建议

非出网环境利用 Spring multipart 临时文件加载恶意 XML:
1. 发送 multipart/form-data 到任意端点, 上传恶意 Spring XML;
2. Spring/Tomcat 保存为 /tmp/tomcat.XXXXX.tmp (随机名);
3. 触发 ClassPathXmlApplicationContext 使用 Ant 通配符:
file:/\${catalina.home}/**/*.tmp 或
file:/tmp/tomcat/**/*.tmp;
4. Spring 加载匹配的 XML 执行 bean 中的恶意代码。
适用: Spring Boot + Embedded Tomcat;
ClassPathXmlApplicationContext 未被完全屏蔽。

原因

multipart 临时文件是 Spring/Tomcat 正常请求处理副产品, 内容攻击者完全控制。Ant pattern 支持 ** 递归通配, 规避了随机文件名问题。核心: 将'可控的本地文件写入'和'任意本地文件加载'组合, 规避出网限制。

领域

java_security

级别

Critical

标签 (逗号分隔)

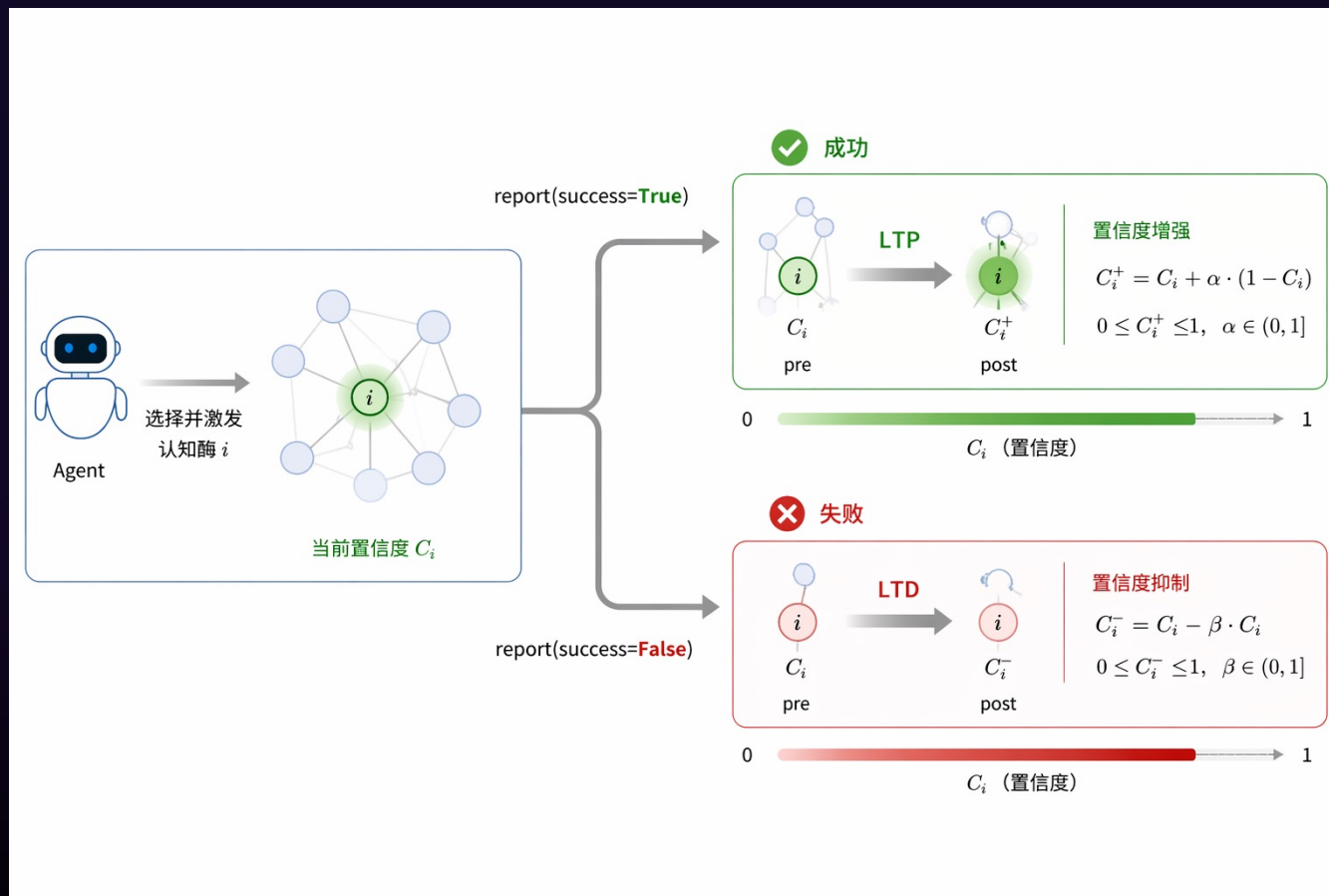
spring-boot, multipart, ant-pattern, rce, no-outbound

取消

保存修改

酶架构基础

Cue [暗示] $\rightarrow$ Routine [行为] $\rightarrow$ Reward [奖励]



进化系统实现

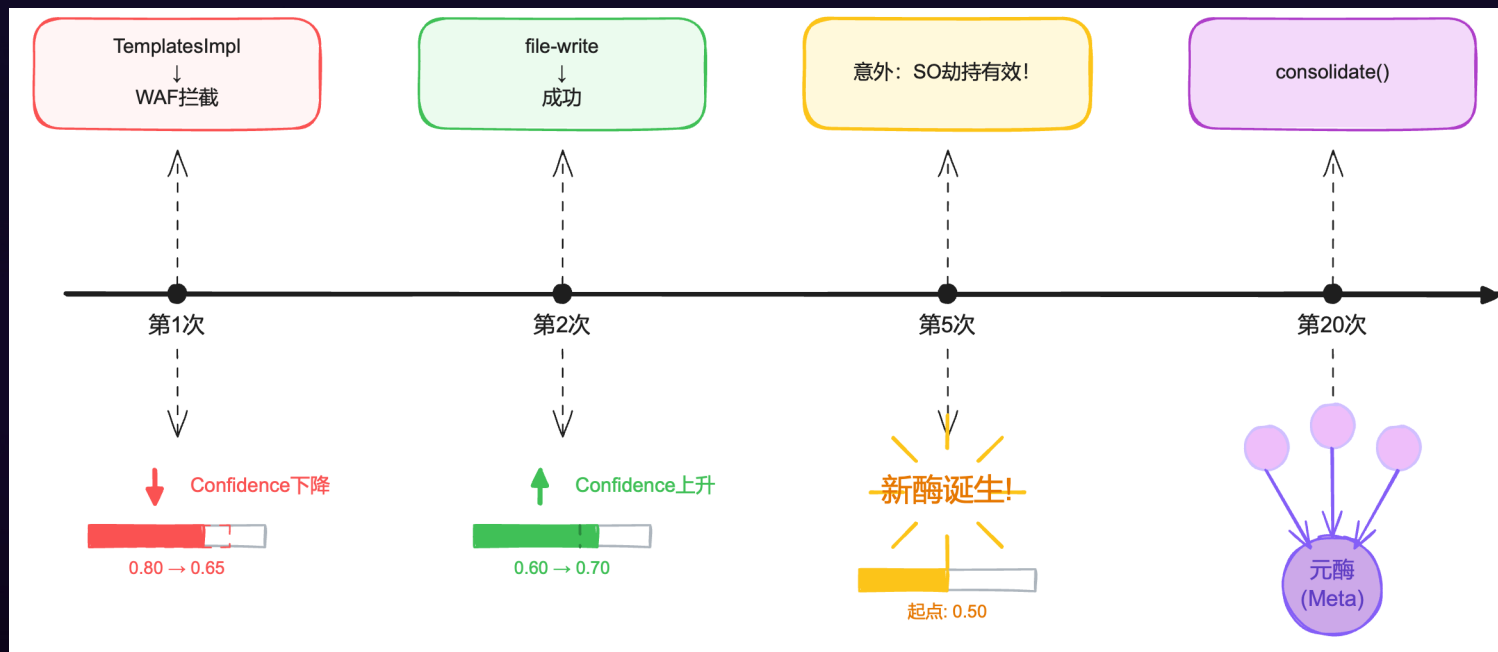
通过历史经验被当前代码特征直接激活



多巴胺误差

学习的信号不是"奖励"本身，而是"奖励与预期的差异"

变化才是进化的催化剂



海马体机制

不引入遗忘和时效，Agent不是人类，不需要遗忘，只要重点。针对记忆质量进行裁剪

Cognitive Enzyme

Dashboard

酶库

隔离审核

进化

搜索

巩固

MCP

向量图谱

设置

Documents

记忆巩固

衰减 → 裁剪 → 合并 → 变异 → 交叉 → 白日梦 → 免疫巡逻 → 规则生成

执行巩固

定时任务

运行中

执行间隔: 6小时

暂停

白日梦+免疫: 开

上次运行: 2026/4/24 16:07:49

巩固结果

共 17 个酶

衰减0
长期未触发的酶降低置信度

休眠0
低精度酶降权休眠

淘汰0
失败的变异体清除

合并0
相似酶泛化合并

变异0
高置信度酶产生变体

交叉0
共触发酶组合新酶

白日梦0
随机酶创意碰撞生成假说

僵尸0
反复失败的酶被隔离

过期0
长期未用的酶被隔离

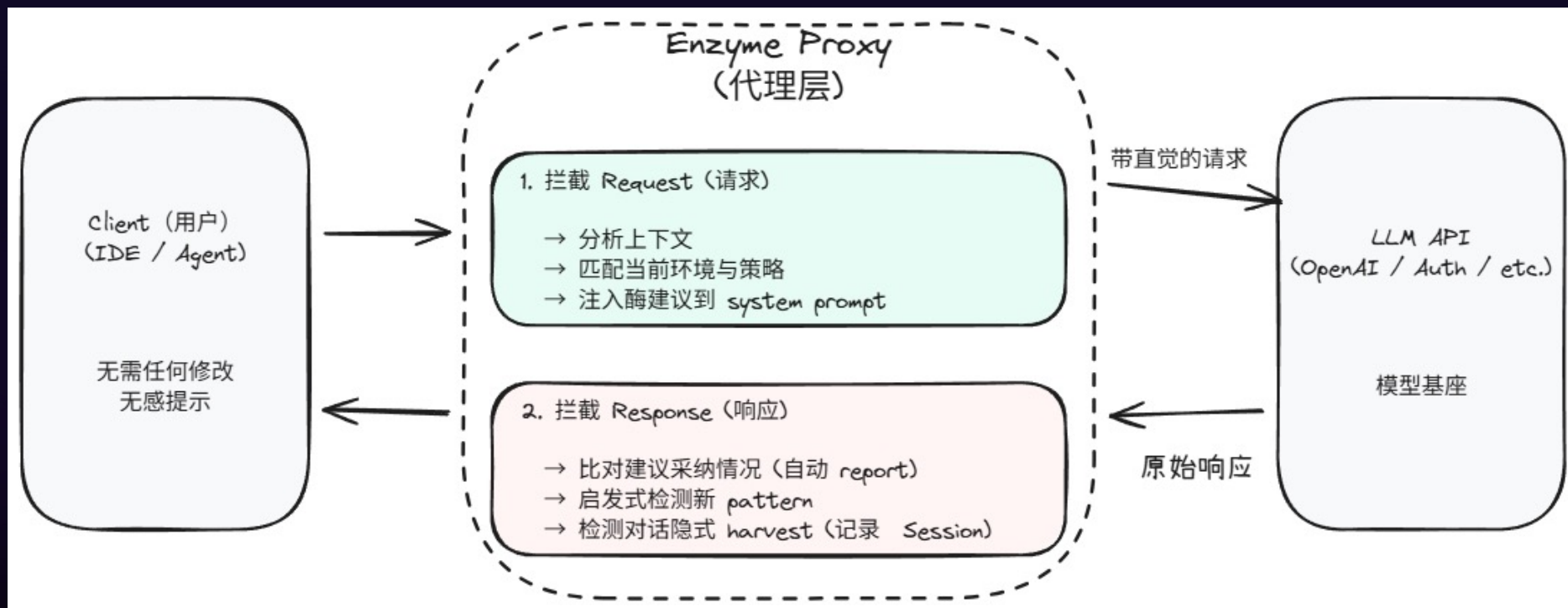
矛盾0
互相矛盾的酶被裁决

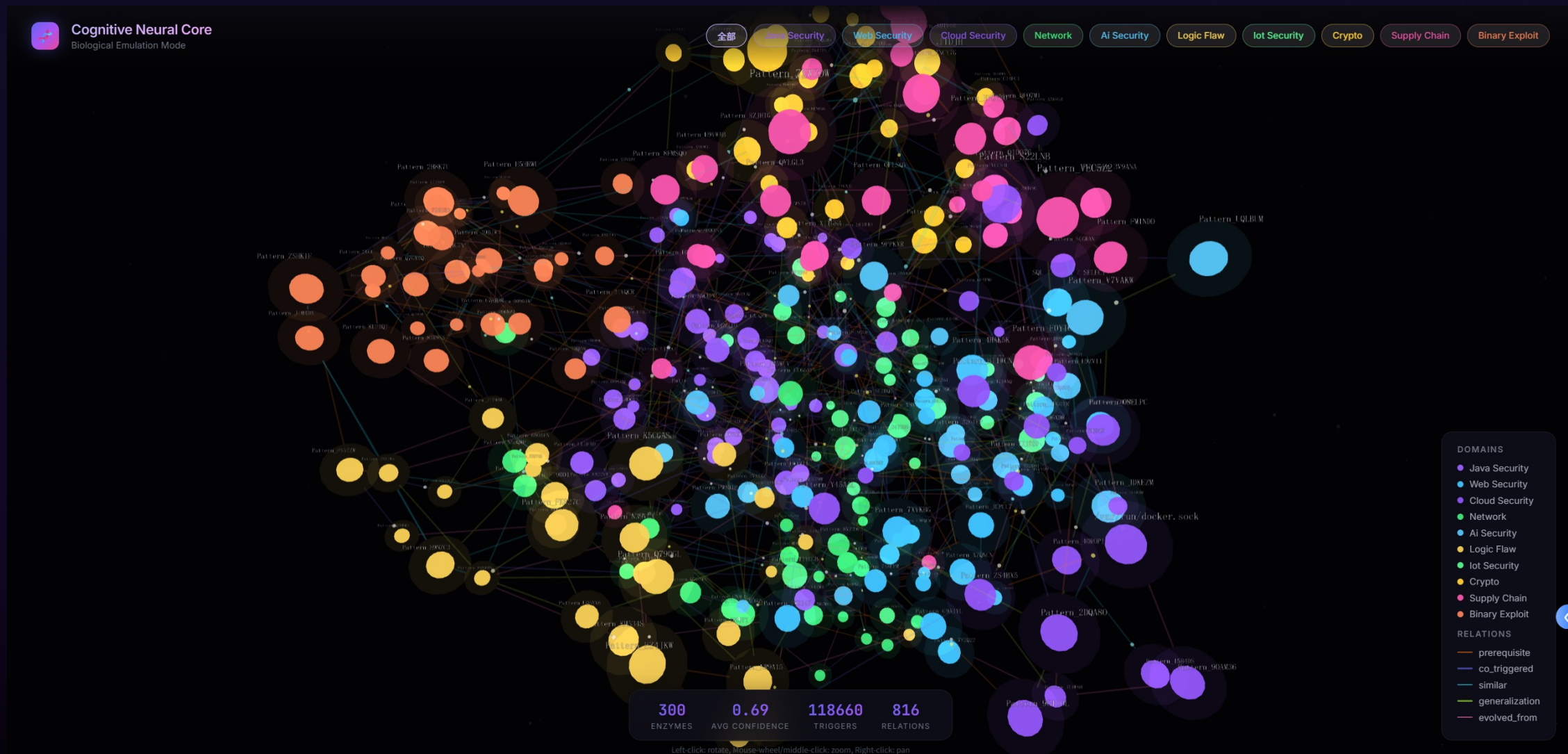
规则0
成熟酶导出为静态规则

变异裁剪: 0

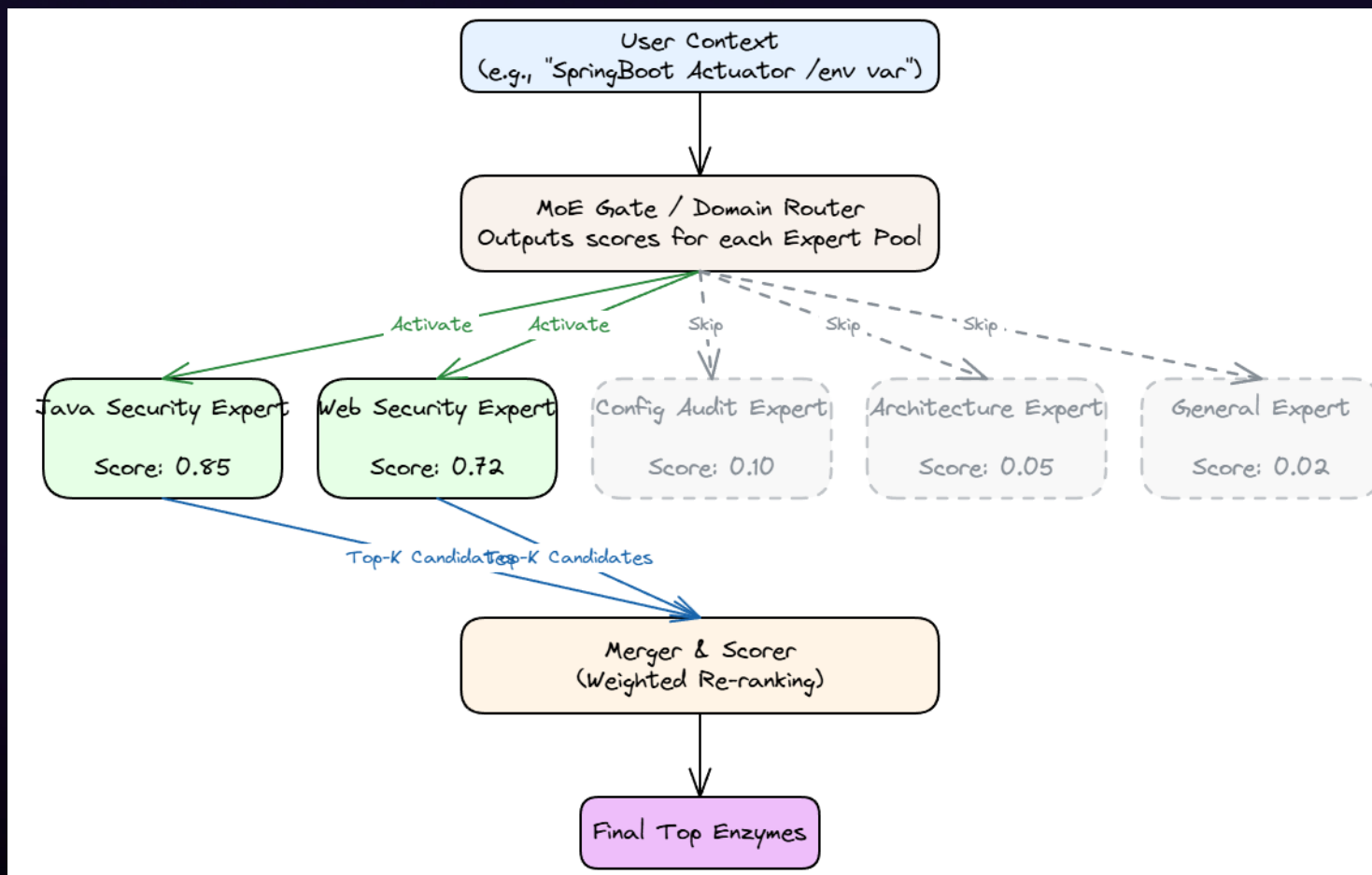
被动注入

不靠大模型"主动搜索", 任务前自动注入 [cache命中可能会下降]





Agent层的MOE框架



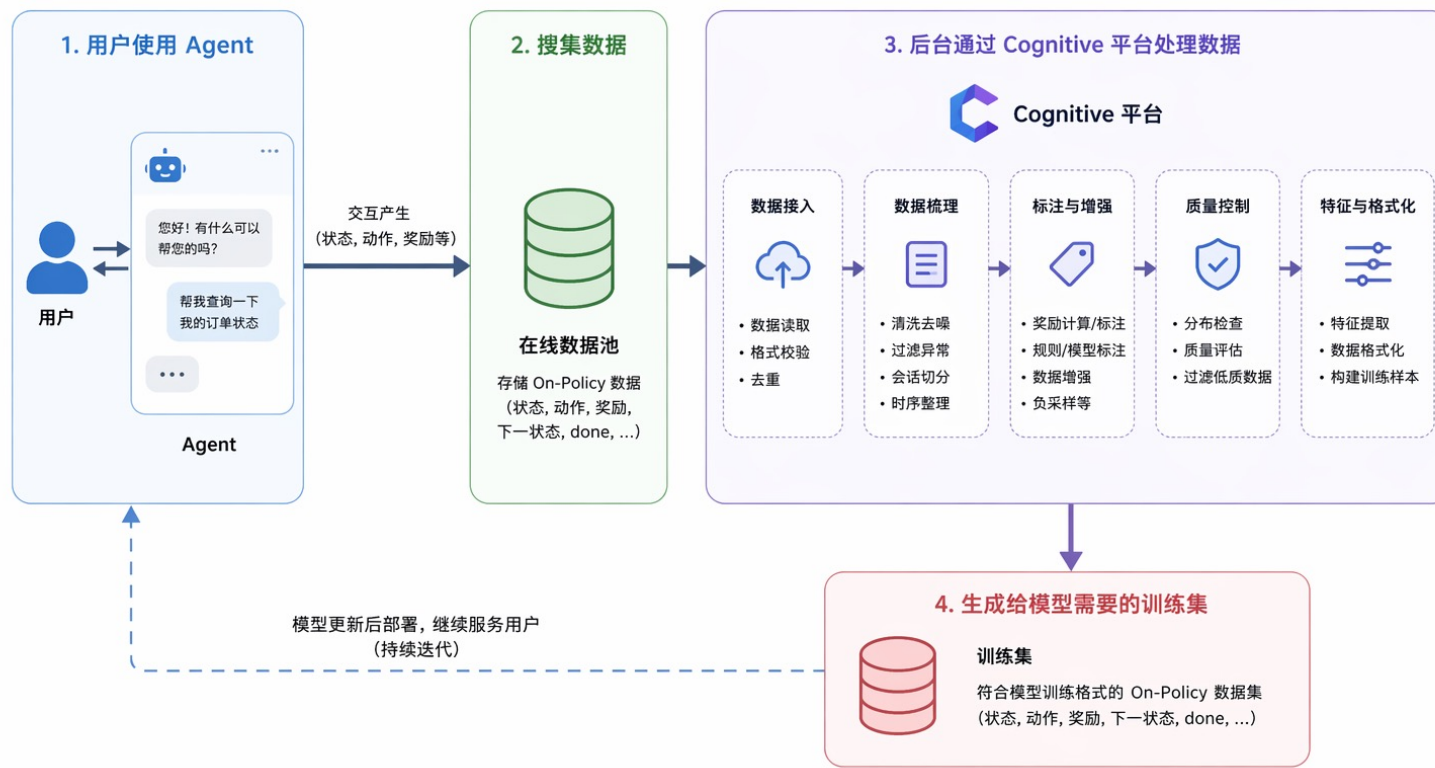
05

未来展望



数据集:

On-Policy 流程图



THANKS

Thanks: Unam4, 柯字辈, java-chians

